



ITAI 1378 • COMPUTER VISION — FINAL PROJECT

PPE Compliance Checker

Automated detection of hard hats, hi-vis vests, and safety goggles — flagging every worker as compliant or non-compliant.

Trilok Kalani • Matthew Jenkins

Tier 2 • mAP@50 0.831 on the six classes the compliance rule uses • ~20 ms/image (T4)

GitHub: github.com/Tikskalani/ITAI-1378-Final_PPE-Compliance-Checker

The Problem

Preventable injuries + compliance liability on the job site

Workers who skip required PPE face some of the most common — and most preventable — worksite injuries: head trauma, struck-by incidents, and eye injuries.

Manual spot-checks don't scale. One safety officer cannot watch every worker continuously, and violations are logged inconsistently, if at all.

The result is both a safety gap and an OSHA-compliance liability.

1

Who cares

Safety officers / EHS managers, superintendents, construction & manufacturing firms, insurers

2

Why it matters

Head, eye & struck-by hazards are leading, largely preventable injuries

3

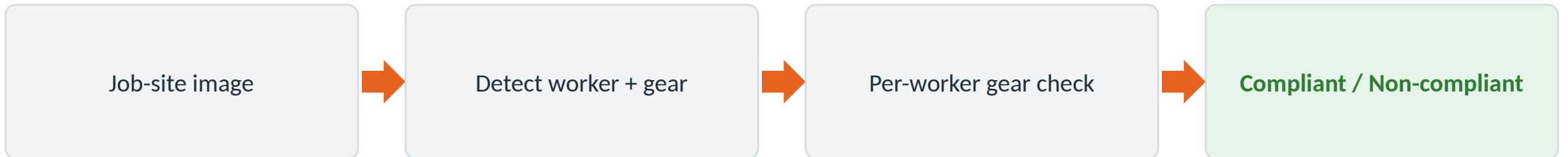
Cost of failure

Direct OSHA citations + injury and liability cost

The Solution

One sentence, one model

An image of a work area goes into a fine-tuned detector that locates each worker and each PPE item, then labels every person “Compliant” or “Non-compliant.”



V2

Next step (V2): extend to a live or recorded video stream — track each worker across frames and log compliance events with timestamps, making it a deployable, portfolio-ready safety-monitoring tool.

Technical Approach

Object detection, done with the fastest reliable tool

1

TECHNIQUE

Multi-class object detection

Locate + identify people and several gear types in one pass.

2

MODEL

YOLO11 (Ultralytics), fine-tuned

Best accuracy-per-training-hour; trains on free Colab GPUs.

3

FRAMEWORK

PyTorch + Ultralytics

Standard, free, Colab-ready; built-in tracking for V2 video.

4

COMPLIANCE LOGIC

Rule-based, per worker

Associate gear to person; require detected hat + vest (goggles configurable).

Why YOLO11: top accuracy-per-training-hour for multi-class detection on a modest budget, and its inference speed is what carries the same model into the real-time V2 capstone without a rewrite.

Data Used

Public, already-labeled datasets — no manual collection

Roboflow Universe — PPE

Thousands of labeled images (a common set: ~19K / 3.6K / 1.9K)
person, hardhat, vest, goggles + NO- variants

SH17

8,099 images · ~75,994 instances
17 PPE + body-part classes (V2 scale-up)

Ultralytics Construction-PPE ← used for V1

1,416 images (1,132 train / 143 val / 141 test)
11 classes incl. helmet, vest, goggles, no_helmet, no_goggle

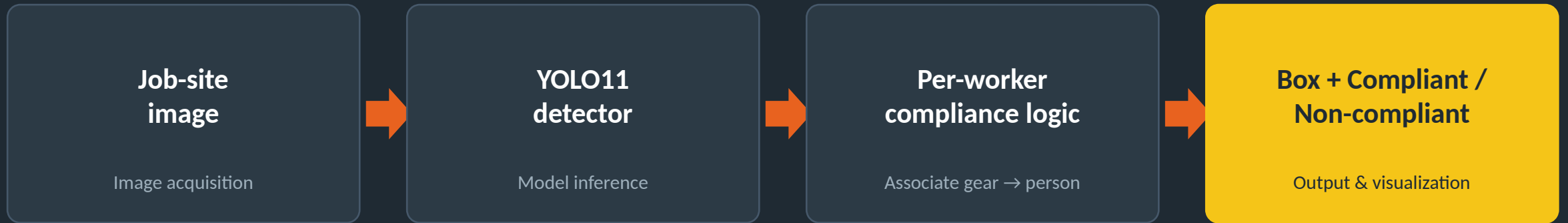
V1 uses Construction-PPE

- 1,416 images total
- 1,132 train / 143 val / 141 test
- Auto-downloads (178 MB), zero setup
- Standardize 640×640, light augmentation

Size: ample for a first fine-tune; scale to SH17 (8,099) for V2.

System Diagram

Input → inference → compliance decision



V2

Capstone: video stream → track workers across frames (ByteTrack) → timestamped compliance log.

Success Metrics

Target vs. measured — on the task the system actually performs

≥ 0.85

mAP@50 (target)

Detection quality across all classes

0.831

mAP@50 achieved (6-class)

The classes the compliance rule uses

0.608

mAP@50 across all 11 classes

Held back by 5 incoherent label classes

Supporting checks

- On the 6 classes the decision uses: precision 0.83 · recall 0.78 · mAP@50-95 0.448. The held-out test split confirms it: 0.824.
- Latency: ~20 ms/image end-to-end on a T4 — the < 1 s/image target met with room to spare.
- The 11-class figure is capped by label defects, not model quality — the evidence is on the next two slides.

Final result YOLO11s · 960px · Tesla T4 · 6-class val 0.831 / test 0.824 · 11-class val 0.608 (V1) → 0.569 (V2). Evaluation images unchanged throughout.

How the Six Weeks Went

The plan vs. what actually happened

1 Set up Colab; downloaded and explored Construction-PPE **Done — 1,416 imgs in YOLO format**

2 Fine-tuned YOLO11s — 40 epochs, ~17 min on a T4 **Done — first real mAP: 0.61**

3 Added per-worker compliance logic **Done — end-to-end labels on images**

4 Evaluated on the held-out val set (143 imgs) **0.61 vs 0.85 target — gap traced to rare no_* classes**

5 Built the demo — 6 annotated sample outputs **Done — samples in results/**

6 Laptop lost — recovered all work from GitHub; docs and slides rebuilt **Version control saved the project**

Challenges & Lessons Learned

What actually happened — and what saved us

CHALLENGE

V2 (960px + oversampling) scored WORSE than V1 — 0.569 vs 0.608

The no_* labels contradict the gear labels

The "none" class was quietly stealing real detections

Laptop lost near the deadline — all local files gone

OUTCOME

Root cause: oversampling duplicated off-domain stock photos. Every no_* class fell; no_boots hit 0.000

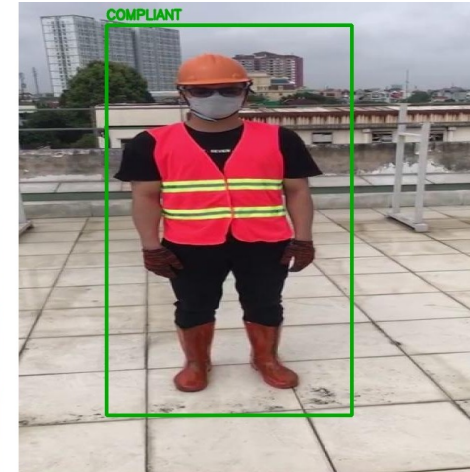
True no_boots is predicted as boots 50% of the time — irreducible error, not a training problem

It absorbed true vest 13%, gloves 8%, helmet 5% — removing it hands those back

Everything recovered from the public GitHub repo — commit early, commit often

System in Action — Sample Outputs

Real outputs — positive-evidence rule, src/predict_compliance.py

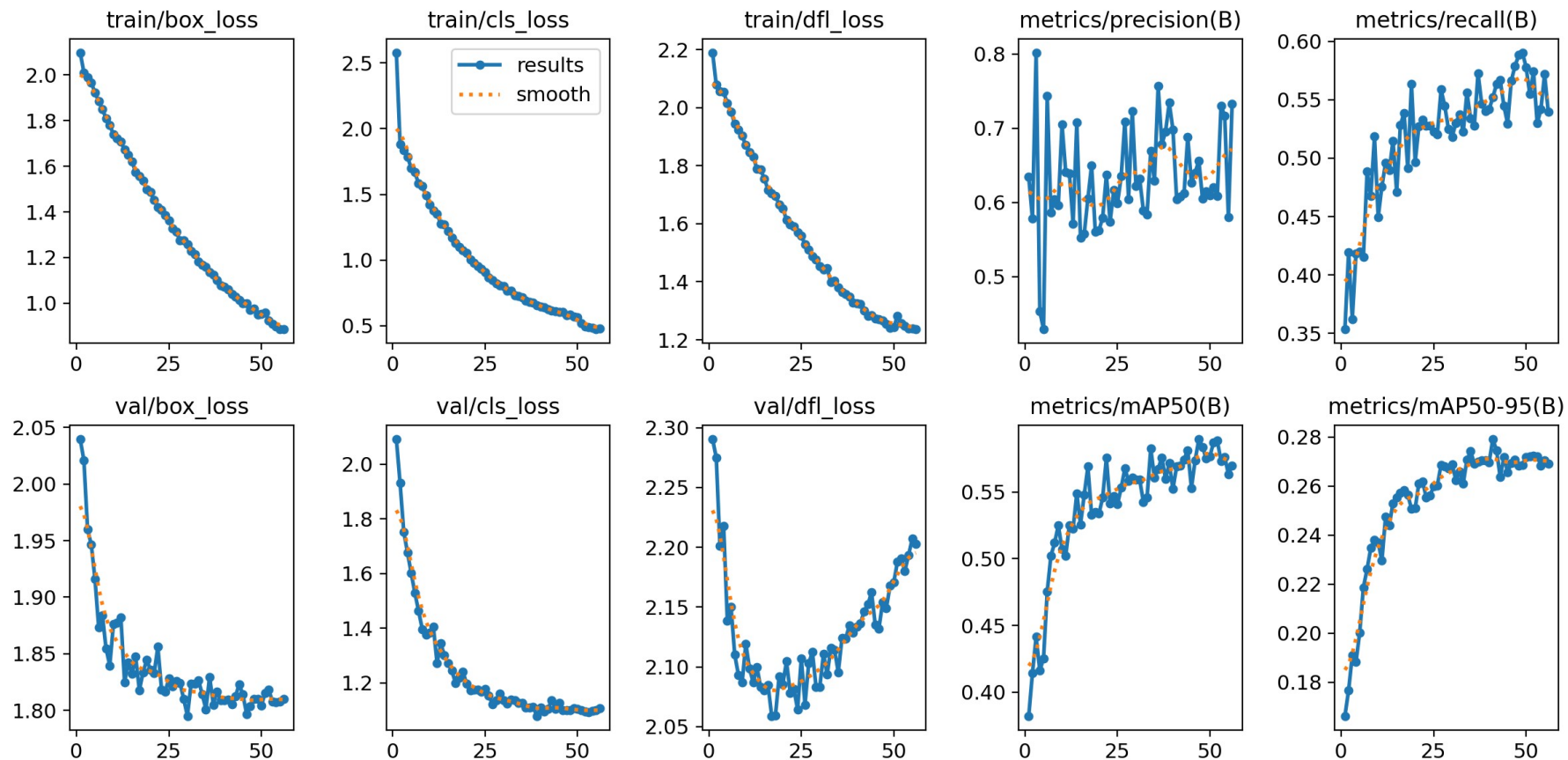


How to read

- Green box = worker judged COMPLIANT
- Red box = NON-COMPLIANT, missing gear listed
- Rule: vest on person, no no_helmet / no_goggle
- All 8 samples in [results/v2_run/](#)

The V2 Experiment

960px + rare-class oversampling — a measured regression



What happened

- 960px helped: helmet +0.023, vest +0.013
- Oversampling backfired — it multiplied off-domain images
- Every no_* class fell; no_boots → 0.000
- 11-class mAP@50: 0.608 → 0.569

Per-Class Results

Six coherent classes kept • five incoherent classes dropped

Per-class mAP@50 — kept (left) vs dropped (right)

Person	0.886	none	0.505
vest	0.863	no_helmet	0.370
gloves	0.817	no_gloves	0.225
goggles	0.815	no_goggle	0.177
helmet	0.826	no_boots	0.000
boots	0.781		

Insight: the six kept classes average 0.831; the five dropped average 0.255. Those five are off-domain stock photos with self-contradictory labels. Dropping them measures the system on the job it performs — and every evaluation image is retained.

